Introduction to Algorithms — Study Notes

1. What is an algorithm?

An **algorithm** is a finite, step-by-step procedure for solving a computational problem. It takes some input, carries out a well-defined sequence of operations, and produces an output in a finite amount of time.

A more formal definition: an algorithm is a finite set of unambiguous instructions which, when followed, accomplish a particular task and always terminate.

Origin of the word

The word comes from the name of the 9th-century Persian mathematician **Muhammad ibn Musa al-Khwarizmi**. His writings described systematic procedures for arithmetic with decimal numbers. When translated into Latin, his name became *Algoritmi*, which gave us "algorithm". The word "algebra" also comes from the title of one of his books.

What is a computational problem?

A computational problem is a question stated precisely enough that a machine could answer it. It has two parts:

- **Input specification** — what data is given (for example, a list of n integers).
- **Output specification** — what result is required and how it relates to the input (for example, the same integers in ascending order).

An algorithm is correct for a problem if, for every valid input (every *instance* of the problem), it produces the required output and stops.

Problem	Input	Output
Sorting	A list of n numbers	The same numbers in ascending order
Searching	A list and a key value	The position of the key, or "not found"
GCD	Two positive integers a and b	Their greatest common divisor
Shortest path	A road map and two cities	The shortest route between them
Maximum	A list of n numbers	The largest number

Everyday analogies

Algorithms are not limited to computers. Any precise procedure that reliably reaches a goal is algorithmic:

- **Long division** — inputs are the dividend and divisor; outputs are the quotient and remainder. Every step is fixed.
- **Multiplying two numbers by hand** — the method taught in school is an algorithm.
- **Directions to a place** — "go straight 200 m, turn left at the signal, second building on the right" is a sequence of clear, finite steps.

A cooking recipe is a common analogy, but it is usually an *imperfect* one. An instruction such as "add a pinch of salt" or "cook until golden" is not precise; two people will do it differently. A true algorithm leaves no such room for interpretation.

Why algorithms matter

- **Efficiency.** Two correct algorithms for the same problem can differ enormously in speed. Searching a sorted list of 1,000,000 items takes up to 1,000,000 checks with linear search but only about 20 with binary search.
- **Problem-solving.** Learning algorithms trains you to break a problem into steps and choose a strategy, which is the core skill in programming.
- **Foundation of computing.** Every program, from a calculator app to a search engine, is built on algorithms.
- **Exams and careers.** Algorithms form a core subject of computer science degrees, a major part of competitive exams such as GATE, the basis of most programming contests, and a central topic in technical job interviews.

2. Characteristics of an algorithm

Not every list of steps is an algorithm. The standard definition, popularised by Donald Knuth in *The Art of Computer Programming*, requires five properties.

Property	Meaning	Violated when…
Input	Takes zero or more well-defined inputs	Required data is never specified
Output	Produces at least one output	The procedure computes nothing usable
Definiteness	Every step is clear and unambiguous	A step says "add a little" or "choose a good value"
Finiteness	Terminates after a finite number of steps	A loop has no stopping condition
Effectiveness	Every step is basic enough to be done exactly, in finite time	A step needs an impossible operation, e.g. "write out every digit of π"

2.1 Input

An algorithm takes **zero or more** inputs. Zero is allowed: an algorithm that prints the first 10 prime numbers needs no input at all. When inputs exist, their type and range must be stated (for example, "n is a positive integer").

2.2 Output

An algorithm must produce **at least one** output, otherwise it serves no purpose. The output must have a clear relation to the input, as fixed by the problem statement.

2.3 Definiteness

Each instruction must have exactly one meaning. "Divide x by y" is definite only if we also say what happens when y = 0. "Sort the list" inside an algorithm is acceptable only if sorting is already a known, defined operation.

2.4 Finiteness

For every valid input, the algorithm must stop after a finite number of steps. A procedure that may run forever, such as `while (true) { print "hello" }`, is not an algorithm. An operating system is designed to run continuously, so by this strict definition it is a *program* but not an algorithm.

2.5 Effectiveness

Each step must be simple enough to be carried out exactly, in principle even by a person with pencil and paper. Operations such as adding two integers or comparing two numbers are effective. "Find the largest prime number" is not, because no such number exists.

Other desirable qualities

Beyond the five essential properties, a *good* algorithm should also have:

- **Correctness** — gives the right output for every valid input.
- **Generality** — solves the whole class of problems, not one special case (sort any list, not only a list of 5 numbers).
- **Efficiency** — uses as little time and memory as possible.
- **Simplicity and clarity** — easy to understand, implement and maintain.
- **Language independence** — can be implemented in any programming language.

Example vs non-example

Algorithm (Euclid's GCD): "While b is not 0, replace (a, b) with (b, a mod b). When b becomes 0, a is the GCD." It has inputs (a, b), an output (the GCD), definite steps, guaranteed termination (b strictly decreases) and simple operations. For a = 40 and b = 24 it gives 8.

Not an algorithm: "Keep guessing numbers until you find the GCD." Guessing is not definite, and there is no guarantee it ever stops.

3. Algorithm vs program

A **program** is also a set of instructions for solving a problem, so the two terms are easily confused. The key distinction: **an algorithm is the design of a solution; a program is its implementation** in a specific programming language for a specific machine.

Basis	Algorithm	Program
SDLC phase	Written in the design phase	Written in the implementation phase
Written by	Designer with domain knowledge	Programmer
Language	Any: natural language, pseudocode, flowchart, mathematical notation	A programming language: C, C++, Java, Python, etc.
Syntax	No strict syntax	Strict syntax; must compile or interpret
Hardware and OS	Independent	Dependent
Evaluated by	Analysis (time and space, before execution)	Testing (running on real inputs)
Executed by	Not executed by a computer; read and reasoned about	Executed by a computer
Termination	Must terminate (finiteness)	May run indefinitely (e.g. an OS or server)

3.1 Design phase vs implementation phase

The **Software Development Life Cycle (SDLC)** is the sequence of phases used to build software:

1. Requirement analysis — what the user needs.
2. **Design** — how the problem will be solved. Algorithms are written here.
3. **Implementation (coding)** — the design is converted into a program.
4. Testing — the program is run to find defects.
5. Deployment and maintenance.

In every branch of engineering, design comes before construction. A bridge is never built by trial and error, demolished and rebuilt until it works. Software is different only because code is cheap to delete and rewrite, which tempts people to skip design. The result is wasted time on programs that are thrown away. Writing the algorithm first means you know exactly what you are building before writing a single line of code.

3.2 Domain knowledge vs programming skill

An algorithm requires understanding of the problem domain. Programming skill alone is not enough:

- Software for **accounting** needs someone who knows accounting rules.
- A **hospital management** system needs input from doctors and hospital administrators.
- A **banking** system needs knowledge of interest calculation, regulations and transactions.

The program is then written by a programmer who knows the language and tools. One person can play both roles: a programmer with domain knowledge acts as the designer while writing the algorithm, and as the programmer while coding it.

3.3 Language of expression

An algorithm can be written in any form, as long as the people who will use it can understand it: simple English statements, pseudocode, a flowchart or mathematical notation. It is often written on paper or in a plain text editor.

A program must follow the exact syntax of a programming language, because a compiler or interpreter has to translate it into machine instructions.

3.4 Dependence on hardware and operating system

An algorithm is an abstract idea and runs on no machine, so it is **independent** of hardware and OS. The same binary search algorithm works on a phone, a laptop or a supercomputer.

A program is **dependent** on its platform: the processor, memory, operating system, compiler and libraries. A Windows executable will not run on Linux without changes or recompilation.

3.5 Analysis vs testing

- An algorithm is **analysed**: we reason about how much time and memory it will need as input grows, without running it.
- A program is **tested**: we run it on sample inputs, check the outputs and measure actual running time.

Section 6 covers this distinction in detail.

4. Ways to represent an algorithm

An algorithm can be expressed in three common forms. All three describe the same logic; they differ in precision and readability.

Form	Strengths	Weaknesses
Natural language	Anyone can read it; no training needed	Can be wordy and ambiguous
Pseudocode	Precise, compact, close to code yet language-free	Needs agreed conventions

Form	Strengths	Weaknesses
Flowchart	Visual; shows branches and loops clearly	Large algorithms become hard to draw and edit

4.1 Natural language

Steps are written as plain numbered sentences.

Example — average of three numbers:

1. Read three numbers a, b and c.
2. Add them and store the result in sum.
3. Divide sum by 3 and store the result in avg.
4. Print avg.
5. Stop.

4.2 Pseudocode

Pseudocode ("false code") uses programming-like structure without the syntax rules of any real language. Common conventions:

- **Header:** `Algorithm Name(parameters)` states the name and inputs.
- **Assignment:** `x ← 5` or `x := 5`.
- **Comments:** `// explanation`.
- **Blocks:** shown by indentation or `begin … end` / `{ … }`.
- **Conditionals:** `if condition then … else …`.
- **Loops:** `for i ← 1 to n do …`, `while condition do …`, `repeat … until condition`.
- **Input/output:** `read x`, `write x` or `return x`.
- **Operators:** relational (<, ≤, =, ≠, ≥, >), logical (and, or, not), arithmetic (+, −, ×, /, mod).
- **Arrays:** `A[i]`, usually indexed from 1 in textbooks.

Example — average of three numbers:

```
Algorithm Average(a, b, c)
{
    sum ← a + b + c
    avg ← sum / 3
    return avg
}
```

4.3 Flowchart

A flowchart draws the algorithm as connected shapes. Each shape has a fixed meaning.

Symbol	Shape	Used for
Terminal	Oval / rounded rectangle	Start and Stop
Input / Output	Parallelogram	Read input, print output

Symbol	Shape	Used for
Process	Rectangle	Calculation or assignment
Decision	Diamond	Yes/No or True/False test
Flow line	Arrow	Direction of control
Connector	Small circle	Joins parts of a chart across a page
Predefined process	Rectangle with double side lines	Call to a sub-procedure

Example — larger of two numbers, as a flowchart reads top to bottom:

1. **Start** (oval)
2. **Read a, b** (parallelogram)
3. **Is a > b?** (diamond)
    - Yes → **Print a** (parallelogram)
    - No → **Print b** (parallelogram)
4. **Stop** (oval)

Choosing a form

Natural language suits explaining an idea to non-programmers. Flowcharts suit short procedures with branching, and teaching. Pseudocode is the standard form in algorithm textbooks and exams, because it is precise enough to analyse and translates directly into code.

5. Writing algorithms: method and solved examples

5.1 Steps to develop an algorithm

1. **Understand the problem.** Restate it in your own words and identify inputs, outputs and constraints.
2. **Work small examples by hand.** Solving two or three instances reveals the pattern.
3. **Choose a strategy.** Decide on an approach: direct computation, repetition, divide and conquer, and so on.
4. **Write the steps.** Express them in pseudocode, keeping each step definite.
5. **Check correctness.** Trace the algorithm on sample inputs, including edge cases (empty list, one element, negative values, duplicates).
6. **Analyse efficiency.** Estimate time and space; look for a better approach if needed.
7. **Implement and test.** Convert to a program and test it on a computer.

5.2 Example: swap two numbers

```
Algorithm Swap(a, b)
{
    temp ← a
    a ← b
    b ← temp
}
```

Trace with a = 5, b = 9: temp = 5, a = 9, b = 5. Three assignments are executed whatever the values, so the time is constant. One extra variable (temp) is used, so the extra space is also constant.

5.3 Example: sum of the elements of an array

```
Algorithm Sum(A, n)
{
    s ← 0
    for i ← 1 to n do
        s ← s + A[i]
    return s
}
```

Trace with A = [4, 7, 1], n = 3: s goes 0 → 4 → 11 → 12, and 12 is returned. The loop body runs n times, so the time grows in proportion to n.

5.4 Example: largest element of an array

```
Algorithm Max(A, n)
{
    max ← A[1]
    for i ← 2 to n do
        if A[i] > max then
            max ← A[i]
    return max
}
```

Trace with A = [12, 45, 7, 89, 23]: max goes 12 → 45 → 45 → 89 → 89. It makes n − 1 comparisons.

5.5 Example: linear search

```
Algorithm LinearSearch(A, n, key)
{
    for i ← 1 to n do
        if A[i] = key then
            return i       // found at position i
    return 0               // not found
}
```

If the key is the first element, one comparison is enough (best case). If it is last or absent, n comparisons are needed (worst case).

5.6 Example: GCD by Euclid's algorithm

```
Algorithm GCD(a, b)
{
    while b ≠ 0 do
    {
        r ← a mod b
        a ← b
        b ← r
    }
    return a
}
```

Trace with a = 40, b = 24:

Step	a	b	r = a mod b
1	40	24	16
2	24	16	8
3	16	8	0
4	8	0	— (stop)

The GCD is 8. The loop always ends because b gets strictly smaller each time and cannot go below 0.

5.7 From algorithm to program

The Max algorithm above, implemented in C:

```
#include <stdio.h>

int findMax(int a[], int n) {
    int max = a[0];              /* C arrays start at index 0 */
    for (int i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];
    return max;
}

int main(void) {
    int a[] = {12, 45, 7, 89, 23};
    printf("%d\n", findMax(a, 5));   /* prints 89 */
    return 0;
}
```

The pseudocode ignored details the program must handle: array indices start at 0 in C, variables need declared types, and output needs a library function. These are implementation concerns; the logic is unchanged.

6. Analysis of algorithms

Analysis means predicting the resources an algorithm needs (mainly **time** and **memory**) as a function of the input size n, without running it. It lets us compare algorithms fairly and pick the best one before writing any code.

6.1 A priori analysis vs a posteriori testing

Basis	A priori analysis (algorithm)	A posteriori testing (program)
When	Before execution	After execution
Method	Mathematical reasoning, counting steps	Running code and measuring
Depends on hardware / compiler	No	Yes

Basis	A priori analysis (algorithm)	A posteriori testing (program)
Result	Approximate, as a growth function, e.g. O(n)	Exact, e.g. 3.2 ms on this laptop
Same on every system	Yes	No; varies between machines
Uses asymptotic notation	Yes	No
Cost	Cheap: pen and paper	Costlier: needs software and hardware
Credit for good speed	The algorithm (designer)	Partly the compiler and hardware

The main focus of algorithm study is a priori analysis, since its conclusions hold on any machine and in any language.

6.2 Time complexity

Time complexity is the amount of work an algorithm does, expressed as a function of input size n. We do not count seconds, which depend on the machine. Instead we count **basic operations** (assignments, comparisons, arithmetic), assuming each takes one unit of time.

Frequency count method: count how many times each statement runs, then add.

Statement in Sum(A, n)	Times executed
s ← 0	1
for i ← 1 to n	n + 1 (the final check fails)
s ← s + A[i]	n
return s	1
Total	**2n + 3**

```
f(n) = 2n + 3 \in O(n)
```

The constant terms and coefficients matter little for large n; what matters is that the work grows linearly with n.

Nested loops multiply. For example:

```
for i ← 1 to n do
    for j ← 1 to n do
        statement
```

The inner statement runs n × n = n^2 times, so this fragment is $O(n^2)$.

6.3 Space complexity

Space complexity is the total memory an algorithm needs as a function of n. It has two parts:

```
S(P) = c + S_P(n)
```

- **c (fixed part):** space for the code, simple variables and constants; independent of the input.
- **S_P(n) (variable part):** space that depends on the input, such as arrays, dynamically allocated memory and the recursion stack.

For Sum(A, n): the array A takes n words, and s, i and n take one word each. Total = n + 3 words, so the space is O(n). The *extra* (auxiliary) space beyond the input is only O(1).

6.4 Best, worst and average case

The same algorithm can take different time on different inputs of the same size. For linear search on n elements:

Case	When it happens	Comparisons	Order
Best	Key is the first element	1	O(1)
Worst	Key is last or absent	n	O(n)
Average	Key equally likely anywhere	about (n + 1) / 2	O(n)

The worst case is used most often, because it gives a guarantee: the algorithm will never be slower than that.

6.5 Asymptotic notations

Asymptotic notation describes how a function grows as n becomes very large, ignoring constants and lower-order terms.

Notation	Name	Meaning
O(g(n))	Big-O	Upper bound: grows no faster than g(n)
Ω(g(n))	Big-Omega	Lower bound: grows at least as fast as g(n)
Θ(g(n))	Big-Theta	Tight bound: grows exactly like g(n)

For f(n) = 2n + 3: f(n) = O(n), f(n) = Ω(n), and so f(n) = Θ(n).

6.6 Common growth rates

Listed from fastest (best) to slowest (worst):

Order	Name	Typical example
O(1)	Constant	Swap two numbers; access A[i]
O(log n)	Logarithmic	Binary search

Order	Name	Typical example
O(n)	Linear	Linear search; sum of an array
O(n log n)	Linearithmic	Merge sort; heap sort
$O(n^2)$	Quadratic	Bubble sort; comparing all pairs
$O(n^3)$	Cubic	Simple matrix multiplication
$O(2^n)$	Exponential	All subsets of a set
O(n!)	Factorial	All orderings (brute-force travelling salesman)

```
1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!
```

For n = 1,000, an O(n log n) algorithm does about 10,000 steps, while an $O(n^2)$ one does 1,000,000. That gap is why choosing the right algorithm matters more than faster hardware.

7. Algorithm design techniques and applications

Most algorithms are built using a small number of general strategies. Recognising which strategy fits a problem is the core skill of algorithm design.

Technique	Core idea	Classic examples
Brute force	Try every possibility directly	Linear search; checking all pairs
Divide and conquer	Split into smaller subproblems, solve each, combine results	Binary search, merge sort, quick sort
Greedy method	At each step take the locally best choice	Kruskal's and Prim's MST, Dijkstra's shortest path, Huffman coding
Dynamic programming	Solve overlapping subproblems once, store and reuse results	0/1 knapsack, longest common subsequence, Floyd–Warshall
Backtracking	Build a solution step by step; undo a step when it cannot lead to a solution	N-Queens, Sudoku, graph colouring
Branch and bound	Explore options while pruning branches that cannot beat the best so far	Travelling salesman, job assignment
Randomised	Use random choices to simplify or speed up	Randomised quick sort

7.1 Brute force

The simplest approach: examine all candidates. It is easy to design and always correct, but often slow. It is a useful starting point and a benchmark to improve on.

7.2 Divide and conquer

Three steps: **divide** the problem into smaller parts, **conquer** each part (usually by recursion), and **combine** the results. Binary search halves the search space each time, giving O(log n). Merge sort splits the list in two, sorts each half and merges them, giving O(n log n).

7.3 Greedy method

Make the choice that looks best right now and never reconsider it. Greedy algorithms are fast and simple, but give the optimal answer only for problems with the right structure. Example: to give change using the fewest coins in Indian currency (₹10, ₹5, ₹2, ₹1), always take the largest coin that fits.

7.4 Dynamic programming

Used when a problem breaks into subproblems that repeat. Each subproblem is solved once and its answer stored in a table. Computing the nth Fibonacci number by plain recursion takes exponential time; storing earlier values cuts it to O(n).

7.5 Backtracking and branch and bound

Both search a tree of possible solutions. Backtracking abandons a path as soon as it breaks a constraint (for example, two queens attacking each other). Branch and bound goes further for optimisation problems: it also abandons a path when its best possible result cannot beat the best solution found so far.

7.6 Applications of algorithms

- **Search engines** — indexing, ranking and retrieving web pages.
- **Navigation and maps** — shortest-path algorithms for routes.
- **Finance** — fraud detection, credit scoring, portfolio optimisation and algorithmic trading.
- **Security** — encryption algorithms such as RSA and AES protect online banking and messaging.
- **E-commerce and media** — recommendation systems for products, videos and music.
- **Data compression** — Huffman coding and similar methods in ZIP, JPEG and MP3 files.
- **Networks** — routing data packets across the internet.
- **Operations and logistics** — scheduling, inventory and delivery route planning.
- **Machine learning and AI** — training models is itself driven by optimisation algorithms.

8. Summary, glossary and practice questions

8.1 Summary

1. An algorithm is a finite, unambiguous, step-by-step procedure for solving a computational problem.
2. It must have input (zero or more), output (one or more), definiteness, finiteness and effectiveness.

3. An algorithm is the design; a program is the implementation. Algorithms are written in the design phase of the SDLC, programs in the implementation phase.
4. Algorithms are usually written by someone with domain knowledge, in any understandable notation, and are independent of hardware and OS. Programs are written by programmers in a programming language and depend on the platform.
5. Algorithms are analysed (a priori); programs are tested (a posteriori).
6. Algorithms can be represented in natural language, pseudocode or flowcharts. Pseudocode is the standard in textbooks and exams.
7. Time complexity counts basic operations as a function of n; space complexity counts memory as $c + S_P(n)$.
8. Worst-case analysis gives a guarantee; asymptotic notation (O, Ω, Θ) describes growth for large n.
9. Common growth order: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$.
10. Major design techniques: brute force, divide and conquer, greedy, dynamic programming, backtracking, branch and bound.

8.2 Glossary

Term	Meaning
Algorithm	Finite, step-by-step procedure that solves a computational problem
Program	Implementation of an algorithm in a programming language
Instance	One specific input to a problem
SDLC	Software Development Life Cycle: requirements, design, implementation, testing, maintenance
Domain knowledge	Understanding of the field the software serves
Pseudocode	Structured, language-independent description of an algorithm
Flowchart	Diagram of an algorithm using standard symbols
Definiteness	Each step has exactly one meaning
Finiteness	The algorithm always stops
Effectiveness	Each step is basic and can be done exactly in finite time
A priori analysis	Estimating cost by reasoning, before execution
A posteriori testing	Measuring a program by running it
Time complexity	Growth of running time with input size

Term	Meaning
Space complexity	Growth of memory use with input size
Frequency count	Number of times each statement executes
Big-O	Upper bound on growth rate
Big-Omega	Lower bound on growth rate
Big-Theta	Tight bound on growth rate

8.3 Practice questions

Short answer (2–3 marks)

1. Define an algorithm and list its five characteristics.
2. Why can an algorithm have zero inputs but not zero outputs?
3. Is an operating system an algorithm? Justify your answer.
4. State three differences between an algorithm and a program.
5. What is pseudocode? Give four conventions used in it.
6. Draw and name the flowchart symbols for decision, process and input/output.
7. Distinguish between a priori analysis and a posteriori testing.
8. Write the space complexity formula and explain both parts.

Long answer (5–10 marks)

9. Compare an algorithm and a program on at least six points, with examples.
10. Explain the role of the design phase in the SDLC and why algorithms are written before programs.
11. Write an algorithm to find the GCD of two numbers using Euclid's method. Trace it for (48, 18) and show it satisfies all five characteristics.
12. Using the frequency count method, find the time complexity of an algorithm that adds two $n \times n$ matrices.
13. Explain best, worst and average case with linear search as the example.
14. Describe four algorithm design techniques with one example each.

Quick check (true or false)

15. An algorithm must be written in a programming language. *(False)*
16. A program's running time is the same on every computer. *(False)*
17. Every step of an algorithm must be unambiguous. *(True)*
18. Big-O notation gives an upper bound on growth. *(True)*
19. $O(n \log n)$ grows faster than $O(n^2)$. *(False)*
20. Analysis of an algorithm is done before it is implemented. *(True)*

Sources

- Algorithms — Encyclopedia.com (origin of the word, five properties, examples)

- Difference between Posteriori and Priori analysis — GeeksforGeeks (a priori vs a posteriori comparison)
- Knuth, D. E., *The Art of Computer Programming*, Vol. 1 (characteristics of an algorithm)
- Horowitz, Sahni and Rajasekaran, *Fundamentals of Computer Algorithms* (pseudocode conventions, frequency count, space complexity S(P) = c + S_P)